

Line-by-Line Code Analysis: agent_engine.py

Complete walk-through of the Detective Agent Response Synthesis Engine

PART 1: MODULE DOCSTRING, IMPORTS & SAFE HELPER FUNCTIONS

Line	Source Code Snippet	Detailed Explanation
1-5	<pre>""" Detective Agent - Response Synthesis Engine Combines AST/code-structure results and RAG results into a readable investigation report. """</pre>	Module-level docstring describing the core purpose of this script: synthesizing AST parsing results and RAG vector search results into a unified report.
7	<pre>from typing import Any, Dict, List</pre>	Imports type hints from Python's standard <code>typing</code> module to enable static type checking for variables, dictionary keys, and list return types.
10-16	<pre>def _safe_text(value: Any, default: str = "Not available") -> str: """Safely converts a value to text..."""</pre>	Defines a private helper function <code>_safe_text</code> to safely convert any object to string and guard against <code>NoneType</code> errors.
17-18	<pre>if value is None: return default</pre>	Checks if <code>value</code> is <code>None</code> . If so, returns the default fallback string ("Not available").
20-21	<pre>text = str(value).strip() return text if text else default</pre>	Converts <code>value</code> to a string, strips whitespace, and returns the cleaned text or the default string if it evaluates to empty.
24-28	<pre>def _get_value(result: Dict[str, Any], key: str, default: str = "Not available",) -> str:</pre>	Defines <code>_get_value</code> to extract string fields safely from result dictionaries without throwing a <code>KeyError</code> .
35-36	<pre>if not isinstance(result, dict): return default</pre>	Ensures the <code>result</code> parameter is actually a dictionary before attempting key lookups.
38	<pre>return _safe_text(result.get(key), default)</pre>	Retrieves <code>key</code> safely using <code>.get()</code> and passes the value into <code>_safe_text()</code> for string sanitation.

PART 2: RESULT NORMALIZATION, DEDUPLICATION & CROSS-REFERENCING

Line	Source Code Snippet	Detailed Explanation
41-42	<pre>def _normalise_results(results: Any) -> List[Dict[str, Any]]:</pre>	Defines <code>_normalise_results</code> to convert raw inputs into a clean list of dictionaries regardless of what the parser returned.
50-57	<pre>if not results: return [] if isinstance(results, dict): results = [results] if not isinstance(results, (list, tuple)): return []</pre>	Handles empty values, wraps single dictionaries inside a list, and rejects invalid non-iterable types.
59-63	<pre>return [result for result in results if isinstance(result, dict)]</pre>	Filters out non-dictionary items, guaranteeing that only dictionary objects are returned in the normalized list.

Line	Source Code Snippet	Detailed Explanation
75-76	<pre>unique = [] seen = set()</pre>	Initializes an ordered list `unique` for clean results and a set `seen` to track unique fingerprints.
78-89	<pre>for result in results: path = _get_value(result, "path", "") ... fingerprint = (path.lower(), details.lowe r(), snippet.lower())</pre>	Extracts key attributes and constructs a tuple fingerprint converted to lowercase.
91-96	<pre>if fingerprint in seen: continue seen.add(fingerprint) unique.append(result)</pre>	Skips entries whose fingerprint already exists in `seen`, adding new ones to both `seen` and `unique`.
101-104	<pre>def _find_related_files(ast_results, rag_r esults) -> List[str]:</pre>	Defines `_find_related_files` to cross-check file paths between AST parser results and RAG search results.
112-123	<pre>ast_files = { ... } rag_files = { ... } return sorted(ast_files.intersection(rag_ files))</pre>	Builds sets of valid file paths for both sources and returns a sorted list of their set intersection.

PART 3: RESPONSE SYNTHESIS & REPORT GENERATION

Line	Source Code Snippet	Detailed Explanation
126	<pre>def generate_agent_response(query, ast_res ults, rag_results):</pre>	Main entry-point function invoked by external agent modules to generate the final Markdown report.
137-142	<pre>ast_results = _normalise_results(ast_resu lts) rag_results = _normalise_results(rag_resu lts) ast_results = _deduplicate_results(ast_re sults) rag_results = _deduplicate_results(rag_re sults)</pre>	Normalizes and deduplicates both incoming data structures to ensure safety and remove duplicate evidence.
144-153	<pre>if not ast_results and not rag_results: return ("🕵️ *Detective Agent:* I searche d...")</pre>	Early return handler: if neither search system found any results, returns a formatted "no evidence found" message.
155-171	<pre>if ast_results: report.append("### 🔍 Code Structure Summ ary") for result in ast_results: ...</pre>	Formats AST structural evidence (type, details, file path) into Markdown bullet points.
177-187	<pre>if rag_results: report.append("\n### 📖 Relevant Code Sni ppets") for result in rag_results: ...</pre>	Formats RAG retrieval evidence (file path and code snippet text block) into Markdown sections.
194-205	<pre>related_files = _find_related_files(ast_r esults, rag_results) if related_files: ...</pre>	Invokes the correlation check and appends a "Correlated Evidence" section if overlapping files were identified.
211-236	<pre>report.append("\n### 🧠 Investigation Sum mary") ... return "\n\n".join(report)</pre>	Assembles summary statistics, appends the cross-check conclusions, joins all report sections with double newlines, and returns the final Markdown string.